# WariMitra — Comprehensive Project Inventory & Technical Baseline

**Document ID:** WM-INV-001

**Classification:** Project Inventory & Architectural Audit

**Methodology:** Reverse-Engineered from Source Code, Configs, Architecture Diagrams, and Test Suites

**Target Audience:** Senior Business Analysts, Technical Product Owners, Solution Architects, Interview Panels

**Workspace Root:** `c:\Users\mustafa\Desktop\WariMitra`

## 1. Executive Identification & Metadata

| Attribute | Project Evidence Detail | Evidence File Path |
|---|---|---|
| **Official Name** | **WariMitra** (also referred to as *VariMitra* in hackathon speaker materials) | `README.md:1`, `ARCHITECTURE.md:1`, `VARIMITRA_SPEAKER_CONTEXT.md:1` |
| **System Tagline** | "Digital Wari Ecosystem" / "The world's first unified digital platform for large-scale religious pilgrimage management" | `README.md:6`, `VARIMITRA_SPEAKER_CONTEXT.md:27` |
| **Business Domain** | GovTech / Public Safety / Crowd Management / Religious Pilgrimage & Humanitarian Logistics | `docs/BRD.md:4`, `ARCHITECTURE.md:5` |
| **Application Type** | Multi-tenant Cross-Platform Hybrid System (Mobile App + Web Command Center + Serverless Backend + Web3 Trust Layer) | `README.md:33-54`, `ARCHITECTURE.md:10-59` |
| **Target Scale** | 500,000 to 1,000,000+ pilgrims (Varkaris), 21 walking days, 250+ km route from Alandi/Dehu/Pune to | `VARIMITRA_SPEAKER_CONTEXT.md:60`, `ARCHITECTURE.md:5` |

route from Alandi/Dehu/Pune to



| Attribute | Project Evidence Detail | Evidence File Path |
| --- | --- | --- |
| | Pandharpur | |
| **Hosting Region** | Firebase `asia-south1` (Mumbai, India) | `functions/src/index.ts:21,123,185`, `ARCHITECTURE.md:190` |
| **Current Architectural Stage** | v2.0 Production-Ready Serverless Architecture (Migrated from earlier v1.0 monolithic Django prototype) | `ARCHITECTURE.md:553`, `Reference files/backend_audit.md:1-81` |

## 2. Business Purpose & Core Value Proposition

The Pandharpur Wari is a 700-year-old annual pilgrimage in Maharashtra where hundreds of thousands of devotees walk over 250 km. Prior to WariMitra, administration, policing, emergency healthcare, and municipal bodies operated in silos using paper logs, uncoordinated radio chatter, and informal WhatsApp groups.

**Primary Operational Objectives:**

1. **Real-time Incident & SOS Triage:** Deliver sub-2-second emergency distress broadcast with exact GPS coordinates directly into police and medical queues (`README.md:23-30`, `functions/src/index.ts:20-115`).
2. **Geospatial Healthcare Dispatch:** Enable pilgrims and field volunteers to locate active medical camps within a dynamic radius (default 10 km) sorted by proximity (`functions/src/index.ts:122-177`).
3. **Crowd Density & Chokepoint Visibility:** Provide district collectors, police, and municipal commissioners with live heatmaps and queue telemetry to prevent stampedes (`docs/FRD.md:41-44`, `frontend/src/app/crowd/`).
4. **Missing Person Reunification:** Structured mobile reporting capturing biometric photos, last-seen GPS, and instant field notification (`mobile/lib/screens/missing_person_screen.dart`, `firestore.rules:111-117`).
5. **Auditable Incident Governance (Web3 Trust Layer):** Eliminate disputes over emergency response delays or altered state records by cryptographically hashing event snapshots and anchoring them

5. delays or altered state records by cryptographically hashing event snapshots and anchoring them onto the Sepolia testnet ledger via zero-ETH self-transactions (`docs/WEB3_ARCHITECTURE.md`, `functions/src/trustLayer/`).

## 3. Technology Stack Inventory

### 3.1 Client Layer: Web Command Center (Next.js)

- **Framework:** Next.js 16.2.12 (with App Router), React 19.0.0, TypeScript 5.3.3 (`frontend/package.json:29,31,48`)
- **Styling & Design System:** TailwindCSS 4.0, PostCSS, Lucide React icons, Glassmorphism dark UI (`frontend/package.json:28,35`, `frontend/src/app/globals.css`)
- **State & Data Fetching:** Zustand 5.0.0, TanStack React Query 5.0.0 (`frontend/package.json:21,36`)
- **GIS & Mapping:** Google Maps JavaScript API (`@googlemaps/js-api-loader: ^2.1.1`, `@googlemaps/markerclusterer: ^2.6.2`) (`frontend/package.json:18-19`)
- **Data Visualization & Analytics:** Recharts 2.10.0 (`frontend/package.json:34`)
- **Virtualization:** `react-window: ^1.8.10` for high-throughput live incident queues (`frontend/package.json:33`)
- **Firebase SDK:** Firebase Client SDK 11.10.0 (`frontend/package.json:25`)

### 3.2 Client Layer: Mobile Application (Flutter)

- **Framework:** Flutter SDK 3.9.2+ / Dart (`README.md:9,58`, `mobile/lib/main.dart`)
- **State Management:** Riverpod (`ARCHITECTURE.md:106`, `mobile/lib/providers/`)
- **Location Services:** `geolocator` plugin with 30-second background location updates and battery level reporting (`mobile/lib/services/`, `VARIMITRA_SPEAKER_CONTEXT.md:155-156`)
- **Mapping:** `google_maps_flutter` (`ARCHITECTURE.md:107`)
- **Localization:** Multi-lingual support prioritizing Marathi (`mr`), Hindi (`hi`), and English (`en`) (`ARCHITECTURE.md:115`, `mobile/lib/screens/home_screen.dart`)
- **Local Persistence:** Firestore offline cache enabled (`ARCHITECTURE.md:111,468`)

### 3.3 Backend & Cloud Services Layer

- **Serverless Compute:** Firebase Cloud Functions running on Node.js 20 runtime, TypeScript 5.3 (`functions/package.json`, `ARCHITECTURE.md:138`)

- **Primary Operational Database:** Cloud Firestore (Serverless NoSQL document database, region: `asia-south1`) (`ARCHITECTURE.md:189-263`, `firestore.rules`)
- **Binary File Storage:** Firebase Cloud Storage with strict owner-based and role-based security rules (`storage.rules:1-35`)
- **Authentication Engine:** Firebase Authentication (Phone SMS OTP, Email/Password, Custom Claims RBAC) (`ARCHITECTURE.md:339-366`)
- **Push Notification Service:** Firebase Cloud Messaging (FCM) using `sendEachForMulticast` (`functions/src/index.ts:192-218`)
- **Geospatial Utility:** `geofire-common` for Geohash generation, bounding-box calculation, and Great-Circle distance sorting (`functions/src/index.ts:3,83,139-174`)

### 3.4 Web3 Trust & Verification Layer

- **Blockchain Network:** Ethereum Sepolia Testnet (`README.md:64`, `docs/WEB3_ARCHITECTURE.md:20`)
- **Integration Library:** Ethers.js v6 (`functions/package.json`, `functions/src/trustLayer/blockchainService.ts`)
- **Transaction Pattern:** Asynchronous 0-ETH self-transactions embedding canonical SHA-256 event hashes into transaction hex data (`docs/WEB3_ARCHITECTURE.md:26-31`)
- **Fail-Safe Mode:** Fallback to local Firestore collection `blockchainProofs` with status `LOCAL_PROOF_ONLY` if RPC is degraded (`docs/WEB3_ARCHITECTURE.md:32-41`, `functions/src/index.ts:253-264`)

## 4. User Roles & Permission Inventory (RBAC)

The system defines **7 distinct operational roles** enforced via Firebase Authentication Custom User Claims (`auth.token.userType`) and verified across Cloud Functions, Firestore Rules, and Frontend UI:

| Role ID | Role Name | Primary Interface | Business Scope & Permissions | Evidence Path |
|---|---|---|---|---|
| `pilgrim` | Pilgrim / Varkari | Flutter Mobile App / Web Guest | View routes, check queues/amenities, trigger SOS, share location with | `firestore.rules:41-51`, `docs/FRD.md:7` |

| Role ID | Role Name | Primary Interface | Business Scope & Permissions | Evidence Path |
|---|---|---|---|---|
| | | | location with family, report missing persons | |
| `volunteer` | Ground Volunteer | Flutter Mobile App | Accept field assistance tasks, submit localized crowd/waste/ incident reports, ground-verify reports | `docs/ FRD.md:8,26-30` |
| `dindi_leader` | Procession Leader | Flutter Mobile App | Manage dindi roster, broadcast text/voice alerts to contingent, monitor member locations | `docs/ FRD.md:9,31-35` |
| `medical_officer` | Medical Officer / EMT | Web Dashboard & Mobile | Manage camp occupancy, log patient encounters, update medicine inventory, receive medical SOS | `firestore.rules :59-67`, `docs/ FRD.md:10` |
| `police_officer` | Police / Traffic Control | Web Command Center | Triage SOS incidents, monitor crowd heatmaps, dispatch patrols, manage route diversions, resolve missing persons | `firestore.rules :77-85,93,115`, `docs/FRD.md:11` |

| Role ID | Role Name | Primary Interface | Business Scope & Permissions | Evidence Path |
|---|---|---|---|---|
| `ngo` / `ngo_coordinator` | Relief NGO Coordinator | Web Dashboard | Register free food/water/blanket distribution centers, update stock levels, match supply to crowd demand | `firestore.rules:148-151`, `docs/FRD.md:12` |
| `admin` / `government` | District Magistrate / Command Admin | Web Command Center | Full system oversight, district analytics, audit log inspection, authority user provisioning, protocol override | `firestore.rules:23,27,38,170`, `docs/FRD.md:13-14` |

## 5. Major Modules & Feature Inventory

### Module 1: Emergency SOS & Rapid Incident Response

- **Hold-to-Trigger UI:** 3-second hold physics button with haptic feedback to prevent accidental triggers (`mobile/lib/screens/sos_screen.dart`, `VARIMITRA_SPEAKER_CONTEXT.md:141`).
- **Backend Validation Pipeline:**
- Idempotency check via client UUID (`functions/src/index.ts:43-56`).
- Rate limiting (maximum 5 requests/minute/user) (`functions/src/index.ts:59-70`).
- Strict geofencing (validates latitude [8.0, 35.0] and longitude [68.0, 97.5]) (`functions/src/index.ts:72-80`).
- **Multicast Dispatch:** Automatically broadcasts FCM notifications to registered police, medical officers, and administrators (`functions/src/index.ts:192-218`).

- **Tamper-Evident Proofing:** Automatically generates SHA-256 hash snapshot and writes to Sepolia / `blockchainProofs` (`functions/src/index.ts:237-264`).

### Module 2: Geospatial Medical Operations

- **Proximity Camp Search:** `getMedicalCamps` callable function queries Geohash bounding boxes to return active camps within 10 km sorted by Haversine distance (`functions/src/index.ts:122-177`).
- **Camp Capacity Telemetry:** Tracks total capacity, current patient count, and available beds (`ARCHITECTURE.md:212-218`, `frontend/src/app/medical/`).
- **Emergency Hotline Routing:** Direct click-to-dial integration for state ambulance service 108 (`IEEE_TEST_CASES.md:53`).

### Module 3: Crowd Density Monitoring & Temple Queue Analytics

- **Queue Telemetry:** Monitors Darshan gates (wait time in minutes, queue headcount, gate status: open/paused/closed) (`ARCHITECTURE.md:225-230`, `frontend/src/app/temple/`).
- **Wait-Time Heuristics:** Mathematical approximation utilizing Little's Law ($L = \lambda W$) (`VARIMITRA_SPEAKER_CONTEXT.md:161`).
- **Heatstroke & Environmental Index:** NOAA Heat Index heuristic mapping temperature/humidity against crowd density (`VARIMITRA_SPEAKER_CONTEXT.md:162`).
- **Digital Pass Booking:** Darshan slot booking mechanism with token issuance and capacity quotas (`IEEE_TEST_CASES.md:57-69`, `firestore.rules:132-135`).

### Module 4: Family Locator & Community Reconnection

- **Family Circles:** 6-character alphanumeric invite codes allowing family units to form ad-hoc tracking circles (`ARCHITECTURE.md:219-224`).
- **Live GPS Stream:** Periodic 30-second location ping with battery percentage to locate vulnerable or separated family members (`mobile/lib/screens/family_locator_screen.dart`, `VARIMITRA_SPEAKER_CONTEXT.md:155-156`).
- **Missing Persons Intake:** Multi-field intake form capturing missing person's name, age, physical description, last-seen coordinates, and photo upload to Firebase Storage (`firestore.rules:111-117`, `mobile/lib/screens/missing_person_screen.dart`).

### Module 5: Civic Amenities & Sanitation Tracking

- **Sanitation Facilities Map:** GIS mapping of public mobile toilets with clean/maintenance status

- **Sanitation Facilities Map:** GIS mapping of public mobile toilets with clean/maintenance status indicators (`firestore.rules:105-110`, `mobile/lib/screens/sanitation_screen.dart`).
- **Crowdsourced Waste Reporting:** Citizen geo-reporting of accumulated solid waste or blocked drains with photo upload for municipal cleaning teams (`firestore.rules:97-103`).
- **Drinking Water & Langar Distribution:** Mapping of water tanker points and NGO-run free food camps (`frontend/src/app/services/`, `frontend/src/app/ngo/`).

### Module 6: Cultural Heritage & Saint Information

- **Spiritual Navigation:** Profiles of Wari saints (Sant Dnyaneshwar, Sant Tukaram) and canonical devotional Abhangs (`firestore.rules:137-146`, `frontend/src/app/heritage/`).
- **Palkhi Route Milestones:** Chronological tracking of daily halting spots (*Mukkam*) and historical resting stages (`mobile/lib/screens/heritage_screen.dart`).

## 6. Firestore Database Architecture Inventory

The database is structured into **12 top-level collections**, subcollections, and singleton analytics documents:

```
Cloud Firestore (asia-south1)
├── users/ (User metadata, custom claims, notification tokens)
│   └── {userId}/
│       ├── pilgrims/profile
│       └── liveLocation/current (30s GPS pings)
├── sosAlerts/ (Emergency distress alerts with geohash, severity, idempotency)
├── medicalCamps/ (Camp coordinates, staffCount, bed capacity)
│   └── {campId}/patients/{patientId} (Clinical encounters)
├── familyGroups/ (Group name, ownerId, 6-char inviteCode, memberIds[])
├── policeStations/ (Jurisdictions, contact, station coordinates)
│   └── {stationId}/patrolUnits/{unitId} (Live patrol telemetry)
├── communityReports/ (Crowdsourced hazard/incident logs)
├── wasteReports/ (Sanitation issue reports with photo URLs)
├── toilets/ (Civic toilet locations, clean/busy status)
├── missingPersons/ (Reports, status: missing/investigating/found)
├── lostFoundItems/ (Catalog of lost physical belongings)
├── templeQueues/ (Gate IDs, waitTimeMinutes, currentCount, status)
├── darshanSlots/ (Time windows, booked quota, max capacity)
├── saints/ & abhangs/ (Cultural audio-text repository)
├── ngoResources/ (Food/water inventory records and distribution centers)
├── servicePoints/ & routes/ (Palkhi route waypoints and relief points)
├── auditLogs/ (Immutable Cloud Function action logs, append-only)
├── blockchainProofs/ (SHA-256 canonical event hashes and Sepolia txHashes)
└── analytics/ (Singleton operational dashboards)
    ├── dashboardMetrics (activeSOS, medicalCamps, totalPilgrims)
    └── sosStats (daily/weekly/monthly breakdown by severity)
```

## 7. Known Technical Limitations & Constraints (Evidence-Based)

| Subsystem | Implemented Status | Limitation / Gap Identified | Evidence Path |
|---|---|---|---|
| **Facial Recognition** | ❌ Not Implemented | Missing person matching is manual intake; no automated OpenCV / ML Kit face matching exists in codebase. | `VARIMITRA_SPEAKER_CONTEXT.md:164` |
| **Crowd Prediction** | ◍ Heuristic Model | Uses deterministic formula (surge multiplier & Little's Law), not deep learning / neural network models. | `VARIMITRA_SPEAKER_CONTEXT.md:160-162` |
| **Local SQLite Queue** | ❌ Not Implemented | Mobile app relies on Firestore's default cache; custom SQLite/Isar offline transactional replay queue is absent. | `VARIMITRA_SPEAKER_CONTEXT.md:165` |
| **IVR / SMS Gateway** | ❌ Not Implemented | System requires mobile internet or Wi-Fi; offline feature phone SMS/IVR fallback is not integrated. | `VARIMITRA_SPEAKER_CONTEXT.md:167-168` |
| **Database Rules Safety** | ⚠ Dev/Test Bypass | `sosAlerts` and `blockchainProofs` rules currently have `allow read, write: if true;` | `firestore.rules:55,180` |

| Subsystem | Implemented Status | Limitation / Gap Identified | Evidence Path |
| --- | --- | --- | --- |
| | | `read, write: if true;` for rapid testing. | |
| **Test Case Automation** | ⚪ Manual Shared Document | Test execution is recorded in a shared Firestore document (`testResults/shared`) via unauthenticated callable endpoints. | `functions/src/index.ts:342-419` |

## 8. Summary for the Business Analyst

WariMitra is not a theoretical concept; it is an engineered, full-stack GovTech platform with functioning frontend command dashboards, mobile pilgrim apps, serverless backend micro-services, and an innovative Web3 audit anchor. The solution directly addresses the operational bottlenecks of mass gatherings through automated event routing, geospatial querying, and multi-agency coordination.